

특별편 2.4 - 다단계 페이지 테이블

남궁명수

다단계 페이지 테이블은 어떻게 동작하는가?

[1] 페이지 테이블의 역할

프로세스가 사용하는 주소는 가상 주소다.

프로그램이 사용하는 주소

= 가상 주소

RAM에서 실제 데이터가 있는 주소

= 물리 주소

CPU는 메모리에 접근하기 전에 가상 주소를 물리 주소로 변환해야 한다.

이때 사용하는 자료구조가 페이지 테이블이다.

가상 주소

↓

페이지 테이블

↓

물리 주소

↓

RAM 접근

Linux의 페이지 테이블은 다음과 같은 다단계 구조로 표현된다.

PGD → P4D → PUD → PMD → PTE → 물리 페이지

[2] 왜 여러 단계로 나누는가?

페이지 테이블을 하나의 거대한 배열로 만들면 사용하지 않는 가상 주소까지 모두 관리해야 한다.

예를 들어, 4KiB 페이지를 사용하는 64비트 시스템에서 하나의 거대한 페이지 테이블을 만들면 엄청난 메모리가 필요하다.

다단계 페이지 테이블은 실제로 사용하는 가상 주소 영역에 대해서만 하위 테이블을 만든다.

PGD

├ 사용하지 않는 영역 → 하위 테이블 없음
├ 사용하지 않는 영역 → 하위 테이블 없음
└ 사용하는 영역

↓

P4D

↓

PUD

↓

PMD

↓

PTE

즉, 다단계 페이지 테이블의 목적은 페이지 테이블 자체가 사용하는 메모리를 줄이는 것이다.

[3] 각 단계에는 무엇이 들어 있는가?

PGD, P4D, PUD, PMD는 일반적으로 다음 단계 페이지 테이블의 물리 주소를 저장한다.

마지막 PTE는 실제 데이터가 들어 있는 물리 페이지 프레임의 주소를 저장한다.

PGD Entry → P4D 테이블 주소

P4D Entry → PUD 테이블 주소

PUD Entry → PMD 테이블 주소

PMD Entry → PTE 테이블 주소

PTE Entry → 실제 물리 페이지 프레임 주소

페이지 테이블에는 프로그램의 실제 데이터가 들어 있는 것이 아니다.

페이지 테이블

= 주소 변환 정보와 접근 권한 저장

물리 페이지

= 프로그램의 실제 데이터 저장

[4] 하나의 페이지 테이블에는 몇 개의 Entry가 있는가?

일반적으로 하나의 페이지 테이블도 4KiB($2^{10} \times 2^2$) 크기의 물리 페이지 하나에 저장된다.

페이지 테이블 Entry 하나의 크기는 8바이트다.

$$4\text{KiB} = 4096\text{바이트}$$

$$4096\text{바이트} \div 8\text{바이트}$$

$$= 512\text{개}$$

$$= 2^9\text{개}$$

따라서 각 단계의 페이지 테이블에는 512개의 Entry가 존재한다.

Page Table

Entry 0
Entry 1
Entry 2
...
Entry 511

512개 중 하나를 선택하려면 9비트가 필요하다.

$$2^9 = 512$$

따라서 가상 주소에는 각 단계에서 사용할 9비트 인덱스가 들어 있다.

[5] 가상 주소의 구성

5단계 페이지 테이블과 4KiB 페이지를 사용한다고 가정하면 가상 주소는 다음과 같이 나뉜다.

PGD 번호	P4D 번호	PUD 번호	PMD 번호	PTE 번호	페이지 내부 위치
9비트	9비트	9비트	9비트	9비트	12비트

각 9비트는 해당 단계 페이지 테이블에서 몇 번째 Entry를 선택할지 나타낸다.

마지막 12비트는 물리 페이지 내부의 위치를 나타낸다.

$$4\text{KiB}$$

$$= 4096\text{바이트}$$

$$= 2^{12}\text{바이트}$$

따라서 4KiB 내부의 특정 바이트를 선택하려면 12비트가 필요하다.

[6] 가상 주소가 물리 주소로 변환되는 과정

프로그램이 특정 가상 주소에 접근한다고 해보자.

```
value = *address;
```

CPU 내부의 MMU는 가상 주소를 여러 부분으로 나눈다.

가상 주소

- └ PGD Index
- └ P4D Index
- └ PUD Index
- └ PMD Index
- └ PTE Index
- └ Offset

[1단계] PGD 찾기

CPU는 현재 프로세스의 최상위 페이지 테이블인 PGD의 물리 주소를 알고 있다.
x86-64에서는 일반적으로 CR3 레지스터가 최상위 페이지 테이블의 물리 주소를 가리킨다.

CR3

↓

현재 프로세스의 PGD

MMU는 PGD 인덱스를 사용해 PGD Entry 하나를 선택한다.

PGD Entry 주소

= PGD 시작 주소 + PGD Index × 8바이트

[2단계] P4D 찾기

PGD Entry가 가리키는 P4D 테이블로 이동한다.
가상 주소의 P4D 인덱스를 사용해 P4D Entry를 선택한다.

P4D Entry 주소

= P4D 시작 주소 + P4D Index × 8바이트

선택된 P4D Entry에는 PUD 테이블의 주소가 들어 있다.

[3단계] PUD 찾기

PUD 인덱스를 사용해 PUD Entry를 선택한다.

PUD Entry → PMD 테이블 주소

[4단계] PMD 찾기

PMD 인덱스를 사용해 PMD Entry를 선택한다.

PMD Entry → PTE 테이블 주소

[5단계] PTE 찾기

PTE 인덱스를 사용해 PTE Entry를 선택한다.
PTE에는 실제 물리 페이지 프레임의 주소가 들어 있다.

PTE → 물리 페이지 프레임

[6단계] 물리 주소 완성

PTE에서 얻은 물리 페이지 프레임의 시작 주소에 가상 주소의 offset을 더한다.

물리 주소

= 물리 페이지 프레임 시작 주소 + Offset

[7] PTE에는 주소만 들어 있는가?

PTE에는 물리 페이지 프레임 주소뿐만 아니라 해당 페이지의 상태와 접근 권한도 들어 있다.

PTE

- └ 물리 페이지 프레임 번호
- └ Present: 현재 RAM에 존재하는가?
- └ Read/Write: 쓰기가 가능한가?
- └ User/Supervisor: 사용자 모드에서 접근 가능한가?
- └ Accessed: 최근 접근된 적이 있는가?
- └ Dirty: 페이지 내용이 수정되었는가?
- └ NX: 실행을 금지할 것인가?

MMU는 주소를 변환하면서 접근 권한도 함께 검사한다.

예를 들어, 읽기 전용 페이지에 쓰기를 시도하면 CPU는 Page Fault를 발생시킨다.

[8] 페이지 테이블은 언제 만들어지는가?

페이지 테이블을 따라가며 주소를 변환하는 것은 CPU의 MMU가 담당한다.
페이지 테이블을 만들고 Entry를 채우는 것은 운영체제 커널이 담당한다.

예를 들어, malloc()을 호출했다고 해보자.

```
char *buffer = malloc(4096);
```

일반적으로 malloc() 직후에는 가상 주소 범위만 확보되고, 실제 물리 페이지는 아직 연결되지 않을 수 있다.

프로그램이 실제로 메모리에 접근한다.

```
buffer[0] = 'A';
```

MMU가 페이지 테이블을 확인했는데 유효한 PTE가 없다면 Page Fault가 발생한다.

```
프로그램이 가상 주소에 접근
↓
MMU가 페이지 테이블 탐색
↓
PTE가 없거나 Present가 0
↓
Page Fault 발생
↓
커널의 Page Fault Handler 실행
↓
물리 페이지 프레임 할당
↓
필요한 페이지 테이블 생성
↓
PTE에 물리 프레임과 권한 기록
↓
중단됐던 명령어 다시 실행
```

[9] 하위 페이지 테이블이 없다면?

커널은 필요한 단계의 페이지 테이블이 없으면 새로 할당한다.

```
PGD는 존재
└─ 필요한 P4D가 없음
    └─ P4D 생성
        └─ 필요한 PUD가 없음
            └─ PUD 생성
                └─ PMD 생성
                    └─ PTE 테이블 생성
                        └─ 물리 페이지 연결
```

하지만 가상 페이지 하나를 만들 때마다 모든 페이지 테이블을 새로 만드는 것은 아니다.

이미 존재하는 중간 페이지 테이블은 같은 주소 범위에 속한 여러 가상 페이지가 함께 사용한다.

```
하나의 PMD Entry
    ↓
하나의 PTE 테이블
    ↓
512개의 PTE
    ↓
최대 512개의 4KiB 물리 페이지 연결
```

따라서 하나의 PTE 테이블은 다음 크기의 가상 주소 범위를 관리할 수 있다.

```
512 × 4KiB
= 2MiB
```

[10] TLB가 필요한 이유

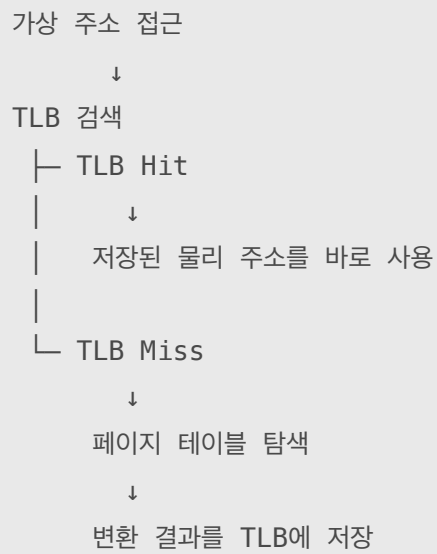
페이지 테이블을 따라가려면 여러 번의 메모리 접근이 필요하다.

```
PGD 읽기
→ P4D 읽기
→ PUD 읽기
→ PMD 읽기
→ PTE 읽기
→ 실제 데이터 읽기
```

이 과정을 매번 수행하면 메모리 접근이 매우 느려진다.
그래서 CPU는 최근에 사용한 가상 주소와 물리 주소의 변환 결과를 TLB에 저장한다.

```
TLB
= 최근 가상 주소 → 물리 주소 변환 결과를 저장하는 CPU 캐시
```

CPU가 가상 주소에 접근하면 먼저 TLB를 확인한다.



따라서 실제로는 대부분 다음 경로를 사용한다.

가상 주소 → TLB → 물리 주소

TLB에 정보가 없을 때만 다단계 페이지 테이블을 탐색한다.

가상 주소

- PGD
- P4D
- PUD
- PMD
- PTE
- 물리 주소

[11] 4단계 시스템의 P4D

Linux는 페이지 테이블을 공통적으로 다음 5단계 구조로 표현한다.

PGD → P4D → PUD → PMD → PTE

하지만 모든 CPU가 실제로 5단계 페이지 테이블을 사용하는 것은 아니다.

4단계 페이지 테이블을 사용하는 x86-64 시스템에서는 P4D가 별도의 실제 테이블로 존재하지 않고 접힌 계층으로 처리될 수 있다.

[Linux의 공통 표현]

PGD → P4D → PUD → PMD → PTE

[실제 4단계 하드웨어]

PGD → PUD → PMD → PTE

이를 P4D가 PGD에 folded되었다고 표현한다.

Linux 커널 코드는 다양한 CPU 구조를 동일한 방식으로 처리하기 위해 5단계 이름을 유지한다.